

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\agent-aefc9eadec7c4893f.jsonl`  
- SHA-256: `e947b7c9cc17fc9e29a20fb2ea3f263ba3a99933f0e9300e9c9166ff91ef305a`  
- Source modified: `2026-06-14T08:55:40+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `subagents`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-14T08:52:18.118Z)

Implement a clean, tested eval-layer SAFETY construct in the badminton-highlight-indexer repo (cwd C:/Users/avidu/Projects/badminton-highlight-indexer): a PER-USER PROMOTION GATE for the personalization feature (the owner adopted personalization as a flag-gated feature on a generalization-first baseline). This is the key safety rail from the strategy analysis.

CONSTRUCT (pure logic, no I/O) ? add backend/eval/per_user_gate.py with a function like ``should_promote_for_user(a_scores, b_scores, *, structural=False, min_n=5, p_threshold=0.05)`` deciding whether to flip a cue/variant ON for ONE user, given that user's PER-VIDEO held-out scores for variant A (control) and B (with the cue). RULES (implement faithfully ? these are the counter-case's mitigations):

1. Promote ONLY IF ALL hold: $n \geq \text{min_n}$ (MINIMUM-N FLOOR ? refuse promotion on too-few videos; default 5) AND the paired ?F1 95% bootstrap CI EXCLUDES 0 AND the paired sign test agrees (majority wins, $p \leq p_threshold$). REUSE backend/eval/eval_stats.py (``paired_delta_ci``, ``paired_sign_test``) ? do NOT reinvent the statistics.
2. STRUCTURAL GUARDS ARE NEVER DISABLED ON 'NO LIFT': if ``structural=True`` (e.g. Gate-A net-crossings, the held-shuttle false-positive killer), the gate must REFUSE to turn it OFF regardless of measured per-user lift ? because its value is in failure modes the user's corpus may NOT yet contain. (i.e. for a structural guard the decision is always 'keep ON'.)
3. Below the floor / insufficient evidence ? decide FALL BACK to the shipped baseline (conservative default), never an unsafe promotion.

Return a small structured result (dataclass or dict): {promote: bool, keep_on: bool, reason: str, n, mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test, floor_met, structural_protected}.

TESTS (tests/test_per_user_gate.py, GPU-free, synthetic per-video deltas): promotes on a clear win above the floor; REFUSES below the floor even with a positive point estimate; REFUSES when CI spans 0 / sign test disagrees (the $n=6$ regime ? use realistic near-null deltas); NEVER disables a structural guard even given a measured LOSS; falls back to baseline below floor.

VERIFY: ``python -m ruff check backend/eval/per_user_gate.py tests/test_per_user_gate.py``;
``python -m mypy backend/eval/per_user_gate.py``; ``python -m pytest tests/test_per_user_gate.py -q``. All pass.

Branch-safe ISOLATED worktree off origin/master: ``git -C C:/Users/avidu/Projects/badminton-highlight-indexer fetch origin`` then ``git -C ... worktree add C:/Users/avidu/Projects/bhi-puser -b feat/per-user-promotion-gate origin/master``. Commit explicit paths (end msg with ``Co-Authored-By: Claude Opus 4.8 (1M context) <noreply@anthropic.com>``); push; ``gh pr create --base master``; DO NOT merge.

Return: PR number/URL, files changed, exact ruff/mypy/pytest results, worktree path.

2. assistant (2026-06-14T08:52:20.593Z)

I'll implement this per-user promotion gate. Let me start by understanding the existing eval-layer code, especially ``eval_stats.py`` which I need to reuse.

3. user (2026-06-14T08:52:23.555Z)

(Bash completed with no output)

4. user (2026-06-14T08:52:23.563Z)

```
1      """Honest small-N evaluation statistics (course-corrections C1 + C3).
2
3      The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
4      a
5      single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
6      training
7      and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
8      ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
9      turns those into an honest summary:
10
11      - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
12      interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
13      many *videos* it wins on (the distribution-free test the repo already leans on:
14      "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
15      noise.
16
17      - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
18      learned
19      producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
20      match, never a held-out window from the same video).
21
22      See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
23      only),
24      deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
25      ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
26      """
27
28      from __future__ import annotations
29
30      import math
31      import random
32      import statistics
33      from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
34
35      Interval = Tuple[float, float]
36      FitPredict = Callable[[List[int], List[int]], List[Any]]
37
38      def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
39                      n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
40          """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
41
42          ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
43          to
44          resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
45          band.
46
47          """
48          vals = [float(v) for v in values]
49          n = len(vals)
50          if n == 0:
51              return (0.0, 0.0)
52          if n == 1:
53              return (vals[0], vals[0])
54          rng = random.Random(seed)
55          means: List[float] = []
56          for _ in range(max(1, n_boot)):
57              means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
58          means.sort()
59          alpha = (1.0 - confidence) / 2.0
60          last = len(means) - 1
61          lo = means[max(0, int(math.floor(alpha * last)))]
```

```

53     hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
54     return (lo, hi)
55
56
57 def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
58                 n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
59     """`{mean, std, n, ci_low, ci_high, confidence}` ? report this, never a bare mean
(C1)."""
60     vals = [float(v) for v in values]
61     n = len(vals)
62     mean = statistics.mean(vals) if vals else 0.0
63     std = statistics.pstdev(vals) if n > 1 else 0.0
64     lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
65     return {"mean": mean, "std": std, "n": float(n),
66           "ci_low": lo, "ci_high": hi, "confidence": confidence}
67
68
69 def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
70     """Two-sided exact sign test over per-fold deltas (B ? A).
71
72     ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
73     excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
74     distribution-free "did B win on enough *videos*" check.
75     """
76     wins = sum(1 for d in deltas if d > eps)
77     losses = sum(1 for d in deltas if d < -eps)
78     ties = len(deltas) - wins - losses
79     n_eff = wins + losses
80     if n_eff == 0:
81         p = 1.0
82     else:
83         k = min(wins, losses)
84         tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
85         p = min(1.0, 2.0 * tail)
86     return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
87           "n_effective": float(n_eff), "p_value": p}
88
89
90 def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
91                    confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
92                    eps: float = 0.0) -> Dict[str, Any]:
93     """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).
94
95     ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
96     by
97     fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
98     sign
99     test agrees ? not when a single mean ticked up.
100     """
101     n = min(len(a_values), len(b_values))
102     deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
103     summary = mean_with_ci(deltas, confidence, n_boot, seed)
104     return {
105         "mean_delta": summary["mean"],
106         "ci_low": summary["ci_low"],
107         "ci_high": summary["ci_high"],
108         "n": float(n),
109         "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
110         "sign_test": paired_sign_test(deltas, eps),
111     }
112
113 def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
114     """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
unit.

```

```

114
115     The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
116     first-seen (deterministic). Groups with no complement (a single group) yield no
fold.
117     """
118     seen: List[Any] = []
119     for g in groups:
120         if g not in seen:
121             seen.append(g)
122     n = len(groups)
123     folds: List[Tuple[List[int], List[int]]] = []
124     for g in seen:
125         test = [i for i in range(n) if groups[i] == g]
126         train = [i for i in range(n) if groups[i] != g]
127         if test and train:
128             folds.append((train, test))
129     return folds
130
131
132     def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
133         """Out-of-fold predictions for honest stacking (C3).
134
135         For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
the
136         other groups and returns predictions for ``test_idx`` (in that order); results are
scattered back to a list aligned to the input items. So a *learned* producer's
137         outputs,
138         when used as arbiter features, never include in-sample predictions ? the leakage
path
139         that inflates measured lift on a tiny corpus. Items in a singleton group stay
``None``
140         (needs >= 2 groups to predict).
141         """
142         n = len(groups)
143         oof: List[Optional[Any]] = [None] * n
144         for train_idx, test_idx in grouped_folds(groups):
145             preds = fit_predict(train_idx, test_idx)
146             for pos, ti in enumerate(test_idx):
147                 if pos < len(preds):
148                     oof[ti] = preds[pos]
149         return oof
150

```

5. assistant (2026-06-14T08:52:28.295Z)

Now let me look at the existing eval module structure and a couple of existing test files to match conventions.

6. user (2026-06-14T08:52:30.363Z)

```

__init__.py
__pycache__
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
eval_stats.py
experiment.py
export_wasb_segments.py

```

```
fusion_audio.py
fusion_compare.py
fusion_golden.py
gemini_refine.py
gt_loader.py
harness.py
metrics.py
rally_seq_proto.py
regression.py
score_calibration.py
segmentation_metrics.py
training.py
---TESTS---
__init__.py
__pycache__
test_annotation_providers.py
test_api_endpoints.py
test_async_jobs.py
test_audio_features.py
test_automated_provider.py
test_base.py
test_calibrate_local.py
test_calibrate_wasb.py
test_calibration.py
test_cleanup_caches.py
test_compiler.py
test_config.py
test_config_wasb.py
test_database.py
test_detector_keying.py
test_distill_local.py
test_eval_annotations.py
test_eval_batch.py
test_eval_classifier.py
test_eval_harness.py
test_eval_metrics.py
test_eval_partial_labels.py
test_eval_regression.py
test_eval_stats.py
test_eval_training.py
test_execution_policy.py
test_experiment.py
test_experiment_honest.py
test_export_wasb_segments.py
test_fusion_compare.py
test_fusion_features.py
test_fusion_golden_telemetry.py
test_fusion_hybrid.py
test_gemini.py
test_gemini_refine.py
test_geometry.py
test_gt_loader.py
test_hashing.py
```

7. user (2026-06-14T08:52:30.476Z)

```
tests\test_eval_stats.py
```

8. user (2026-06-14T08:52:32.712Z)

```
1      """Tests for backend.eval.eval_stats (course-corrections C1 + C3).
2
3      Pure/offline + deterministic (the bootstrap takes a fixed seed). Pins: CIs bracket the
4      mean and degrade sanely at tiny n; the sign test's exact p-value; leave-one-group-out
5      folds; and that out-of-fold predictions are genuinely out of fold (no leakage).
```

```

6     """
7     import pytest
8
9     from backend.eval.eval_stats import (
10         bootstrap_ci,
11         grouped_folds,
12         mean_with_ci,
13         out_of_fold_predictions,
14         paired_delta_ci,
15         paired_sign_test,
16     )
17
18
19     def test_bootstrap_ci_is_deterministic_and_brackets_mean():
20         vals = [0.5, 0.6, 0.7, 0.55, 0.65, 0.6]
21         lo1, hi1 = bootstrap_ci(vals, seed=0)
22         lo2, hi2 = bootstrap_ci(vals, seed=0)
23         assert (lo1, hi1) == (lo2, hi2) # same seed ? identical
24         assert lo1 <= sum(vals) / len(vals) <= hi1
25
26
27     def test_bootstrap_ci_degenerate():
28         assert bootstrap_ci([]) == (0.0, 0.0)
29         assert bootstrap_ci([0.42]) == (0.42, 0.42)
30         assert bootstrap_ci([0.5, 0.5, 0.5]) == (0.5, 0.5) # no spread ? point interval
31
32
33     def test_mean_with_ci_shape():
34         out = mean_with_ci([0.4, 0.6, 0.5], seed=1)
35         assert out["mean"] == pytest.approx(0.5)
36         assert out["n"] == 3.0
37         assert out["ci_low"] <= out["mean"] <= out["ci_high"]
38
39
40     def test_sign_test_all_wins_exact_p():
41         res = paired_sign_test([0.1, 0.2, 0.05, 0.3, 0.15, 0.2]) # 6 wins
42         assert res["wins"] == 6.0 and res["losses"] == 0.0
43         assert res["p_value"] == pytest.approx(2 * 0.5 ** 6) # 0.03125
44
45
46     def test_sign_test_ties_and_mixed():
47         res = paired_sign_test([0.0, 0.0, 0.0])
48         assert res["ties"] == 3.0 and res["n_effective"] == 0.0 and res["p_value"] == 1.0
49         mixed = paired_sign_test([0.1, -0.1, 0.2])
50         assert mixed["wins"] == 2.0 and mixed["losses"] == 1.0
51
52
53     def test_paired_delta_ci_detects_consistent_improvement():
54         a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54]
55         b = [0.60, 0.66, 0.69, 0.61, 0.70, 0.63] # B beats A every fold
56         out = paired_delta_ci(a, b, seed=0)
57         assert out["mean_delta"] > 0.0
58         assert out["sign_test"]["wins"] == 6.0
59         assert out["ci_excludes_zero"] is True
60
61
62     def test_grouped_folds_leave_one_group_out():
63         groups = ["m1", "m1", "m2", "m2", "m3"]
64         folds = grouped_folds(groups)
65         assert len(folds) == 3
66         train0, test0 = folds[0] # first-seen group m1 held out
67         assert test0 == [0, 1] and train0 == [2, 3, 4]
68         assert grouped_folds(["only"]) == [] # single group ? no fold
69
70

```

```

71     def test_out_of_fold_predictions_are_out_of_fold():
72         groups = ["a", "a", "b", "b"]
73         y = [0, 0, 1, 1]
74
75         def fit_predict(train_idx, test_idx):
76             # "Model" = mean label of the training groups; returned for each test item.
77             m = sum(y[i] for i in train_idx) / len(train_idx)
78             return [m] * len(test_idx)
79
80         oof = out_of_fold_predictions(groups, fit_predict)
81         # Group a's OOF uses only group b (mean 1.0) and vice-versa ? never its own labels.
82         assert oof == [1.0, 1.0, 0.0, 0.0]
83

```

9. assistant (2026-06-14T08:52:35.551Z)

Now I understand the conventions well. Let me also check the eval `\_\_init\_\_.py` and how mypy/ruff are configured in this repo.

10. user (2026-06-14T08:52:36.480Z)

```

1     """Rally-detection evaluation harness.
2
3     Scores the pipeline's detected rallies against hand-annotated `[start, end)`
4     ground-truth rally timestamps, using temporal interval matching. See
5     docs/EVALUATION.md for the annotation format and CLI usage.
6
7     - `metrics`      ? pure interval-matching math (IoU, matching, P/R/F1, boundary error).
8     - `annotations` ? load ground-truth rally CSVs.
9     - `harness`      ? read predictions from the DB and score them.
10    """
11

```

11. user (2026-06-14T08:52:38.046Z)

```

ls: cannot access 'pyproject.toml': No such file or directory
ls: cannot access 'setup.cfg': No such file or directory
ls: cannot access '.ruff.toml': No such file or directory
mypy.ini
ruff.toml
---
77082f4 feat(eval): wire run-telemetry black box into fusion_golden extractor (#140)
ec5bbe0 chore(eval): track experiment_leaderboard.json (fusion P3/P4 honest LOVO records) (#139)
74c1303 docs(quality): log fusion P3+P4 measured ?F1 on the n=6 golden corpus (#137)

```

12. assistant (2026-06-14T08:52:40.442Z)

Note: my local fetched `origin/master` head is `77082f4` (#140), newer than the snapshot in the prompt. Let me check the lint configs to match style.

13. user (2026-06-14T08:52:40.916Z)

```

1     # Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would
2     # bury blame history for zero behavior value).
3     #
4     # Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle
5     # warnings (W) + bugbear (B). Deferred with intent: I (import-sort) and UP
6     # (pyupgrade) ? both are large mechanical diffs; adopt via a dedicated PR.
7     target-version = "py38"
8
9     # scratch/ = throwaway debugging scripts that may hit real APIs; it is also
10    # excluded from pytest collection (see pytest.ini).
11    extend-exclude = ["scratch"]
12

```

```

13     [lint]
14     extend-select = ["W", "B"]
15
16     [lint.per-file-ignores]
17     # Entrypoint scripts deliberately run setup (load_dotenv, logging config,
18     # sys.path) before importing backend modules ? import order is load-bearing.
19     "backend/main.py" = ["E402"]
20     "scripts/run_phase3_eval.py" = ["E402"]
21     "scripts/run_process_and_stitch.py" = ["E402"]
22

```

14. user (2026-06-14T08:52:41.368Z)

```

1     [mypy]
2     # Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean
3     # modules and pins an explicit quarantine list below ? not a typing crusade.
4     # Ratchet: when you touch a quarantined module, try removing its entry.
5     ignore_missing_imports = True
6     # backend/ is a namespace package (no root __init__.py) ? required so mypy
7     # maps files to backend.* module names instead of duplicating top-level names.
8     explicit_package_bases = True
9
10    # --- Quarantine (known-loose modules, each with a reason) ---
11
12    # Optional module globals (db_instance/global_config set only in main());
13    # properly fixed by the deferred app-factory/lifespan refactor (BACKLOG).
14    [mypy-backend.main]
15    ignore_errors = True
16
17    # Legacy demo segmenter (fabricated metadata; coverage-omitted too).
18    [mypy-backend.pipeline.segmenters.motion]
19    ignore_errors = True
20
21    # yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code
22    # to detectors/person_detector.py (which IS type-clean, no quarantine). What remains
23    # here is pre-existing orchestration debt unrelated to that move (implicit-Optional
24    # process_video defaults, the Success|Failure ABC-override mismatch, a Future[...]
25    # annotation) ? ratchet those out in a focused typing pass, not here.
26    # (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)
27    [mypy-backend.pipeline.segmenters.yolo_hybrid]
28    ignore_errors = True
29
30    [mypy-backend.pipeline.segmenters.gemini]
31    ignore_errors = True
32
33    # Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.
34    [mypy-backend.pipeline.segmenters.base]
35    ignore_errors = True
36

```

15. assistant (2026-06-14T08:52:44.554Z)

I have everything I need. Now let me set up the isolated worktree off `origin/master`, then write the files.

16. user (2026-06-14T08:52:46.713Z)

Preparing worktree (new branch 'feat/per-user-promotion-gate')
branch 'feat/per-user-promotion-gate' set up to track 'origin/master'.
HEAD is now at 77082f4 feat(eval): wire run-telemetry black box into fusion_golden extractor (#140)

17. user (2026-06-14T08:52:50.220Z)


```
feat/per-user-promotion-gate
77082f4 feat(eval): wire run-telemetry black box into fusion_golden extractor (#140)
--- origin/master..HEAD (should be empty) ---
```

18. assistant (2026-06-14T08:52:52.853Z)

Clean base, no sibling commits. Now let me write the per-user gate module.

19. assistant (2026-06-14T08:53:28.902Z)

```
"""Per-user promotion gate ? the safety rail for flag-gated personalization.
```

```
The owner adopted personalization as a flag-gated feature on a generalization-first
baseline: a cue/variant B may be flipped ON for one user only when that user's own
held-out evidence honestly clears the same small-N bar the global eval uses ? otherwise we
fall back to the shipped baseline (variant A). This module is the decision logic for that
flip; it is pure (no I/O) and reuses backend.eval.eval_stats for every statistic
(paired_delta_ci + paired_sign_test) so the math is never reinvented.
```

The counter-case this guards against (and its mitigations, implemented here):

1. **Minimum-N floor.** Personalizing off 1-2 of a user's videos is the small-N-lies regime (see `eval_stats` C1). Refuse to promote below `min_n` videos ? point estimates from too few held-out videos are noise, not signal.
2. **Honest lift, not a single mean.** Promote only when the paired ?F1 95% bootstrap CI **excludes 0** and the paired sign test agrees (majority of videos win, $p \leq \text{threshold}$). This is the global promotion bar applied per user.
3. **Structural guards are never disabled on "no lift".** A **structural** guard (e.g. Gate-A net-crossings ? the held-shuttle false-positive killer) earns its keep through failure modes a given user's corpus may not contain yet. Measuring "no per-user lift" on a corpus that simply lacks those failure cases must NEVER turn the guard OFF. For `structural=True` the decision is therefore **always "keep ON"**, independent of the measured delta.

Below the floor / insufficient evidence ? fall back to the shipped baseline (the conservative default), never an unsafe promotion. All additive, pure, deterministic (the bootstrap seed is threaded through). Nothing here changes existing eval behaviour; callers opt in.

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
from typing import Any, Dict, Optional, Sequence
```

```
from backend.eval.eval_stats import paired_delta_ci, paired_sign_test
```

```
__all__ = ["PromotionDecision", "should_promote_for_user"]
```

```
@dataclass
```

```
class PromotionDecision:
```

```
    """Structured result of a per-user promotion decision.
```

- `promote` ? flip variant B ON for this user (only ever `True` for a non-structural cue that cleared the full bar).
- `keep_on` ? the cue/guard should be ON for this user. For a structural guard this is always `True` (it is never disabled on "no lift"); for a non-structural cue it equals `promote`.
- `reason` ? short human-readable explanation of the decision.
- the remaining fields surface the evidence (n, mean delta, CI bounds and the two tests) so a caller can log/audit exactly why a user was or wasn't personalized.

```
"""
```

```
promote: bool
keep_on: bool
reason: str
n: int
```

```

mean_delta: float
ci_low: float
ci_high: float
ci_excludes_zero: bool
sign_test: Dict[str, float] = field(default_factory=dict)
floor_met: bool = False
structural_protected: bool = False

def as_dict(self) -> Dict[str, Any]:
    """Plain-dict view (for JSON logging / telemetry)."""
    return {
        "promote": self.promote,
        "keep_on": self.keep_on,
        "reason": self.reason,
        "n": self.n,
        "mean_delta": self.mean_delta,
        "ci_low": self.ci_low,
        "ci_high": self.ci_high,
        "ci_excludes_zero": self.ci_excludes_zero,
        "sign_test": self.sign_test,
        "floor_met": self.floor_met,
        "structural_protected": self.structural_protected,
    }

def should_promote_for_user(
    a_scores: Sequence[float],
    b_scores: Sequence[float],
    *,
    structural: bool = False,
    min_n: int = 5,
    p_threshold: float = 0.05,
    confidence: float = 0.95,
    n_boot: int = 2000,
    seed: int = 0,
    eps: float = 0.0,
) -> PromotionDecision:
    """Decide whether to flip variant B ON for ONE user from that user's held-out scores.

    ``a_scores`` / ``b_scores`` are the user's **per-video** held-out F1 (or any score) for
    variant A (control / shipped baseline) and variant B (with the cue), aligned by video
    (same order). The decision rules (faithful to the module docstring):

    1. ``structural=True`` ? the guard is **never disabled on "no lift"**: return
       ``keep_on=True, promote=False`` regardless of the measured per-user delta. Its value is
       in failure modes the user's corpus may not yet contain.
    2. Otherwise promote **iff ALL** hold ? the minimum-N floor (``n >= min_n``), the paired
       ?F1 bootstrap CI **excludes 0**, and the paired sign test agrees (majority of videos
       win, ``p_value <= p_threshold``). Any miss ? fall back to the shipped baseline.

    Statistics come straight from ``eval_stats`` (``paired_delta_ci`` / ``paired_sign_test``);
    deterministic given ``seed``. Returns a :class:`PromotionDecision`.
    """
    n = min(len(a_scores), len(b_scores))
    floor_met = n >= min_n

    # --- Evidence (reuse eval_stats; never reinvent the statistics) -----
    if n == 0:
        # No paired videos at all ? nothing to test. Surface an empty-but-honest result.
        sign_test: Dict[str, float] = paired_sign_test([], eps)
        mean_delta, ci_low, ci_high, ci_excludes_zero = 0.0, 0.0, 0.0, False
    else:
        delta = paired_delta_ci(
            a_scores, b_scores,
            confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,

```

```

    )
    mean_delta = float(delta["mean_delta"])
    ci_low = float(delta["ci_low"])
    ci_high = float(delta["ci_high"])
    ci_excludes_zero = bool(delta["ci_excludes_zero"])
    sign_test = delta["sign_test"]

sign_p = float(sign_test.get("p_value", 1.0))
sign_wins = float(sign_test.get("wins", 0.0))
sign_losses = float(sign_test.get("losses", 0.0))
# The sign test "agrees" with a promotion only if it is significant AND the majority
# leans the *winning* way (more B-wins than B-losses) ? a significant net *loss* must
# never satisfy the promotion bar.
sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)

# --- Rule 1: structural guards are never disabled on "no lift" -----
if structural:
    return PromotionDecision(
        promote=False,          # nothing to "promote" ? it ships ON by design
        keep_on=True,           # ALWAYS kept ON, regardless of measured per-user lift
        reason=(
            "structural guard kept ON regardless of measured per-user lift "
            "(value is in failure modes this user's corpus may not yet contain)"
        ),
        n=n,
        mean_delta=mean_delta,
        ci_low=ci_low,
        ci_high=ci_high,
        ci_excludes_zero=ci_excludes_zero,
        sign_test=sign_test,
        floor_met=floor_met,
        structural_protected=True,
    )

# --- Rule 2: minimum-N floor (refuse before even looking at lift) -----
if not floor_met:
    return PromotionDecision(
        promote=False,
        keep_on=False,          # fall back to the shipped baseline (variant A)
        reason=(
            f"below minimum-N floor (n={n} < min_n={min_n}); "
            "falling back to shipped baseline"
        ),
        n=n,
        mean_delta=mean_delta,
        ci_low=ci_low,
        ci_high=ci_high,
        ci_excludes_zero=ci_excludes_zero,
        sign_test=sign_test,
        floor_met=False,
        structural_protected=False,
    )

# --- Rule 2 (cont.): honest lift ? CI excludes 0 AND sign test agrees -----
if ci_excludes_zero and sign_agrees:
    promote = True
    reason = (
        f"promoted: n={n} >= {min_n}, ?F1 95% CI "
        f"[{ci_low:.4f}, {ci_high:.4f}] excludes 0, "
        f"sign test {int(sign_wins)}-{int(sign_losses)} (p={sign_p:.4f} <= {p_threshold})"
    )
else:
    promote = False
    misses = []
    if not ci_excludes_zero:

```

```

        misses.append(f"?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] spans 0")
    if not sign_agrees:
        misses.append(
            f"sign test does not agree ({int(sign_wins)}-{int(sign_losses)}), "
            f"p={sign_p:.4f}"
        )
    reason = (
        "insufficient evidence (" + "; ".join(misses) + "); "
        "falling back to shipped baseline"
    )

    return PromotionDecision(
        promote=promote,
        keep_on=promote,          # non-structural: keep_on tracks promote exactly
        reason=reason,
        n=n,
        mean_delta=mean_delta,
        ci_low=ci_low,
        ci_high=ci_high,
        ci_excludes_zero=ci_excludes_zero,
        sign_test=sign_test,
        floor_met=True,
        structural_protected=False,
    )

```

20. user (2026-06-14T08:53:28.917Z)

File created successfully at: C:/Users/avidu/Projects/bhi-puser/backend/eval/per_user_gate.py
(file state is current in your context ? no need to Read it back)

21. assistant (2026-06-14T08:53:33.894Z)

I have an unused import (`Optional`). Let me remove it to keep ruff clean.

22. user (2026-06-14T08:53:35.062Z)

The file C:/Users/avidu/Projects/bhi-puser/backend/eval/per_user_gate.py has been updated successfully. (file state is current in your context ? no need to Read it back)

23. assistant (2026-06-14T08:53:38.465Z)

Now let me write the tests. I need to construct realistic synthetic per-video deltas. Let me first verify what delta magnitudes produce a CI that excludes 0 vs spans 0, and confirm the sign-test p-value thresholds for n=6.

24. user (2026-06-14T08:53:49.512Z)

```

CLEAR WIN n=8: mean_delta=0.1187 ci=[0.1150,0.1225] excl0= True wins= 8.0 p=0.0078
N=3 POSITIVE: mean_delta=0.1400 ci=[0.1200,0.1500] excl0= True wins= 3.0 p=0.2500
N=6 deltas: [0.01, -0.02, 0.02, -0.02, 0.01, -0.01]
N=6 NEAR-NULL: mean_delta=-0.0017 ci=[-0.0150,0.0117] excl0= False wins= 3.0 losses= 3.0
p=1.0000
LOSS n=8: mean_delta=-0.1000 ci=[-0.1000,-0.1000] excl0= True wins= 0.0 losses= 8.0 p=0.0078

```

25. assistant (2026-06-14T08:53:54.437Z)

All the regimes behave exactly as needed:

- **Clear win n=8**: CI excludes 0, sign test 8-0 p=0.0078 ? promotes.
- **n=3 positive point estimate**: below floor (would refuse on floor regardless).
- **n=6 near-null**: CI spans 0, sign test 3-3 p=1.0 ? refuses.
- **Loss n=8**: CI excludes 0 but it's a *net loss* (0-8). For a non-structural cue, `sign\_agrees` requires wins>losses, so this correctly does NOT promote. For a structural guard, this is the key case ? must keep ON despite the measured loss.

Now let me write the tests.

26. assistant (2026-06-14T08:54:21.093Z)

```
"""Tests for backend.eval.per_user_gate ? the per-user personalization promotion gate.
```

GPU-free, fully synthetic per-video scores, deterministic (the bootstrap seed is threaded through ``should\_promote\_for\_user``). Pins the four safety properties of the counter-case mitigations:

- promotes a non-structural cue on a clear, above-floor win;
- refuses below the minimum-N floor *even with* a positive point estimate (small-N lies);
- refuses when the ?F1 CI spans 0 / the sign test disagrees (the realistic n=6 near-null);
- **never** disables a structural guard, even given a measured per-user LOSS;
- falls back to the shipped baseline (``keep\_on=False``) whenever it refuses to promote.

```
"""  
from backend.eval.per_user_gate import PromotionDecision, should_promote_for_user
```

```
def test_promotes_clear_win_above_floor():  
    """n=8, B beats A on every video by a healthy margin ? promote ON."""  
    a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54, 0.56, 0.53]  
    b = [0.62, 0.67, 0.71, 0.63, 0.70, 0.66, 0.69, 0.65]  
    d = should_promote_for_user(a, b, min_n=5, seed=0)  
    assert isinstance(d, PromotionDecision)  
    assert d.promote is True  
    assert d.keep_on is True          # non-structural: keep_on tracks promote  
    assert d.floor_met is True  
    assert d.ci_excludes_zero is True  
    assert d.mean_delta > 0.0  
    assert d.sign_test["wins"] == 8.0 and d.sign_test["losses"] == 0.0  
    assert d.structural_protected is False
```

```
def test_refuses_below_floor_even_with_positive_point_estimate():  
    """n=3 with a strongly positive point estimate is still REFUSED ? small-N lies.  
  
    The floor is checked before lift, so even though this CI would 'exclude 0' the gate must  
    fall back to the baseline rather than personalize off too few videos.  
    """  
    a = [0.50, 0.55, 0.60]  
    b = [0.65, 0.70, 0.72]          # B wins all 3, big positive mean delta  
    d = should_promote_for_user(a, b, min_n=5, seed=0)  
    assert d.mean_delta > 0.0        # point estimate genuinely looks good ...  
    assert d.promote is False        # ... but we refuse on too-few videos  
    assert d.keep_on is False        # ? fall back to shipped baseline  
    assert d.floor_met is False  
    assert d.n == 3  
    assert "floor" in d.reason.lower()
```

```
def test_refuses_n6_near_null_ci_spans_zero_and_sign_test_disagrees():  
    """The realistic n=6 regime: tiny mixed deltas ? CI spans 0 and sign test is 3-3."""  
    a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54]  
    b = [0.51, 0.53, 0.62, 0.50, 0.59, 0.53]  # deltas ~ +.01, -.02, +.02, -.02, +.01, -.01  
    d = should_promote_for_user(a, b, min_n=5, seed=0)  
    assert d.floor_met is True        # above the floor ? refusal is on the evidence, not n  
    assert d.ci_excludes_zero is False  
    assert d.sign_test["wins"] == 3.0 and d.sign_test["losses"] == 3.0  
    assert d.promote is False  
    assert d.keep_on is False        # fall back to baseline  
    assert d.structural_protected is False
```

```
def test_refuses_when_only_sign_test_fails():
```

```

"""Guard the AND: even a CI that excludes 0 must not promote if the sign test disagrees.

A measured net *loss* (B worse on every video) has a CI that excludes 0 but leans the
wrong way ? the gate must not read 'significant' as 'promote'.
"""
a = [0.60, 0.62, 0.65, 0.61, 0.63, 0.64, 0.66, 0.62]
b = [0.50, 0.52, 0.55, 0.51, 0.53, 0.54, 0.56, 0.52] # B loses every video
d = should_promote_for_user(a, b, min_n=5, seed=0)
assert d.ci_excludes_zero is True # CI clears 0 ...
assert d.mean_delta < 0.0 # ... but on the losing side
assert d.sign_test["wins"] == 0.0 and d.sign_test["losses"] == 8.0
assert d.promote is False
assert d.keep_on is False

def test_structural_guard_never_disabled_even_on_measured_loss():
    """structural=True ? always keep_on, regardless of a measured per-user LOSS.

    Gate-A net-crossings is the held-shuttle FP killer; its value is in failure modes a given
    user's corpus may not contain. A measured loss on such a corpus must NOT turn it OFF.
    """
    a = [0.60, 0.62, 0.65, 0.61, 0.63, 0.64, 0.66, 0.62]
    b = [0.50, 0.52, 0.55, 0.51, 0.53, 0.54, 0.56, 0.52] # B clearly worse here
    d = should_promote_for_user(a, b, structural=True, min_n=5, seed=0)
    assert d.keep_on is True # NEVER disabled on 'no lift' / loss
    assert d.promote is False # nothing to 'promote' ? ships ON by design
    assert d.structural_protected is True
    assert d.mean_delta < 0.0 # evidence is surfaced, decision ignores it
    assert "structural" in d.reason.lower()

def test_structural_guard_kept_on_below_floor_too():
    """A structural guard stays ON even when the user has too few videos to measure lift."""
    a = [0.60, 0.62]
    b = [0.40, 0.41] # tiny corpus, B looks worse
    d = should_promote_for_user(a, b, structural=True, min_n=5, seed=0)
    assert d.keep_on is True
    assert d.promote is False
    assert d.structural_protected is True

def test_empty_input_falls_back_to_baseline():
    """No paired videos ? no evidence ? fall back to the shipped baseline (non-structural)."""
    d = should_promote_for_user([], [], min_n=5, seed=0)
    assert d.n == 0
    assert d.floor_met is False
    assert d.promote is False
    assert d.keep_on is False

def test_decision_as_dict_round_trips_fields():
    """as_dict() exposes the full structured result for telemetry/audit logging."""
    a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54, 0.56, 0.53]
    b = [0.62, 0.67, 0.71, 0.63, 0.70, 0.66, 0.69, 0.65]
    d = should_promote_for_user(a, b, min_n=5, seed=0)
    out = d.as_dict()
    for key in (
        "promote", "keep_on", "reason", "n", "mean_delta", "ci_low", "ci_high",
        "ci_excludes_zero", "sign_test", "floor_met", "structural_protected",
    ):
        assert key in out
    assert out["promote"] is True and out["keep_on"] is True

```

File created successfully at: C:/Users/avidu/Projects/bhi-puser/tests/test_per_user_gate.py
(file state is current in your context ? no need to Read it back)

28. assistant (2026-06-14T08:54:23.126Z)

Now let me run the verification: ruff, mypy, and pytest.

29. user (2026-06-14T08:54:24.837Z)

```
===== RUFF =====  
All checks passed!  
EXIT=0
```

30. user (2026-06-14T08:54:26.863Z)

```
===== MYPY =====  
Success: no issues found in 1 source file  
EXIT=0
```

31. assistant (2026-06-14T08:54:33.809Z)

Ruff and mypy both clean. Now pytest.

32. user (2026-06-14T08:54:35.447Z)

```
===== PYTEST =====  
..... [100%]  
8 passed in 0.12s  
EXIT=0
```

33. assistant (2026-06-14T08:54:37.772Z)

All 8 tests pass. Let me run a quick broader sanity check to confirm I didn't break the existing eval_stats suite (since I import from it), then commit.

34. user (2026-06-14T08:54:39.521Z)

```
..... [100%]  
8 passed in 0.09s
```

35. assistant (2026-06-14T08:54:41.995Z)

Now let me re-verify the branch state before staging, then commit explicit paths.

36. user (2026-06-14T08:54:43.606Z)

```
feat/per-user-promotion-gate  
77082f4 feat(eval): wire run-telemetry black box into fusion_golden extractor (#140)  
--- origin/master..HEAD (must be empty) ---  
--- status ---  
?? backend/eval/per_user_gate.py  
?? tests/test_per_user_gate.py
```

37. assistant (2026-06-14T08:54:46.021Z)

Clean: only my two new files, no sibling commits on the branch. Staging explicit paths and committing.

38. user (2026-06-14T08:54:46.860Z)

warning: in the working copy of 'backend/eval/per_user_gate.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_per_user_gate.py', LF will be replaced by CRLF the

next time Git touches it
A backend/eval/per_user_gate.py
A tests/test_per_user_gate.py

39. user (2026-06-14T08:54:58.681Z)

```
[feat/per-user-promotion-gate 8680dfe] feat(eval): per-user promotion gate for flag-gated personalization
2 files changed, 332 insertions(+)
create mode 100644 backend/eval/per_user_gate.py
create mode 100644 tests/test_per_user_gate.py
```

40. user (2026-06-14T08:55:06.130Z)

```
8680dfe feat(eval): per-user promotion gate for flag-gated personalization
--- push ---
remote:
remote: Create a pull request for 'feat/per-user-promotion-gate' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/feat/per-user-promotion-gate
remote:
branch 'feat/per-user-promotion-gate' set up to track 'origin/feat/per-user-promotion-gate'.
To https://github.com/avidullu/badminton-highlight-indexer.git
* [new branch]      feat/per-user-promotion-gate -> feat/per-user-promotion-gate
```

41. assistant (2026-06-14T08:55:08.094Z)

Pushed. Now creating the PR.

42. user (2026-06-14T08:55:23.821Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/141>

43. assistant (2026-06-14T08:55:26.234Z)

PR created. Let me do a final confirmation of the PR state and clean up nothing (worktree stays for the owner).

44. user (2026-06-14T08:55:28.197Z)

```
{"baseRefName":"master","headRefName":"feat/per-user-promotion-gate","number":141,"state":"OPEN","title":"feat(eval): per-user promotion gate for flag-gated personalization","url":"https://github.com/avidullu/badminton-highlight-indexer/pull/141"}
```

45. assistant (2026-06-14T08:55:40.378Z)

Done. The per-user promotion gate is implemented, tested, verified green, committed to an isolated worktree, pushed, and opened as a PR (not merged).

Summary

****PR:**** #141 ? <https://github.com/avidullu/badminton-highlight-indexer/pull/141> (OPEN, base `master`, head `feat/per-user-promotion-gate`, NOT merged)

****Worktree:**** `C:/Users/avidu/Projects/bhi-puser` (isolated, branched off `origin/master` @ `77082f4` ? note this is newer than the prompt's git-status snapshot; the prompt named #127 as HEAD but origin/master had advanced to #140. Branch contained ONLY my one commit `8680dfe` before push.)

****Files changed (2, +332):****

```
- `backend/eval/per_user_gate.py` ? `should_promote_for_user(a_scores, b_scores, *,
structural=False, min_n=5, p_threshold=0.05, confidence=0.95, n_boot=2000, seed=0, eps=0.0)`
returning a `PromotionDecision` dataclass (with `.as_dict()` for telemetry). Pure, no I/O,
```


deterministic.

- `tests/test\_per\_user\_gate.py` ? 8 GPU-free synthetic tests.

****The three rules, as implemented:****

1. ****Minimum-N floor**** ? refuses below `min\_n` (default 5) before even looking at lift.
2. ****Honest lift**** ? promotes only if `n >= min\_n` AND CI excludes 0 AND sign test agrees (`p <= p\_threshold` AND `wins > losses`). Statistics are ****reused**** from `backend/eval/eval\_stats.py` (`paired\_delta\_ci`, `paired\_sign\_test`) ? not reinvented. The `wins > losses` guard ensures a significant **net loss** (CI excludes 0 on the wrong side) never promotes.
3. ****Structural guards never disabled on "no lift"***** ? `structural=True` always returns `keep\_on=True` regardless of measured per-user lift (validated even against a measured loss and below the floor).

Below floor / insufficient evidence ? `keep\_on=False` (falls back to shipped baseline).

****Exact verification results (all pass):****

- `python -m ruff check backend/eval/per\_user\_gate.py tests/test\_per\_user\_gate.py` ? `All checks passed!` (exit 0)
- `python -m mypy backend/eval/per\_user\_gate.py` ? `Success: no issues found in 1 source file` (exit 0)
- `python -m pytest tests/test\_per\_user\_gate.py -q` ? `8 passed in 0.12s` (exit 0)

Also confirmed the existing `tests/test\_eval\_stats.py` suite stays green (8 passed) since the module imports from it.

One note for the owner: per the project's merge working-agreement, this is an eval-layer engineering construct rather than a product/strategy/licensing change, but I left it unmerged as instructed ? it's ready for your review/CI on PR #141.